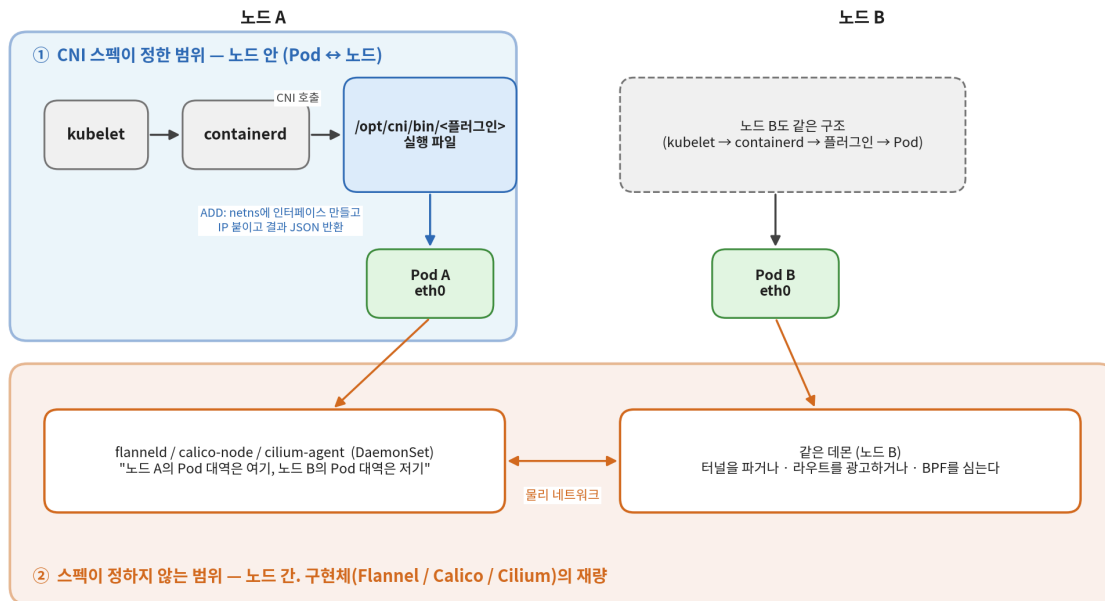


CNI 생태계



쿠버네티스 네트워크 모델의 요구: "모든 Pod가 NAT 없이 서로 도달할 것" → ①만으로는 안 되고 ②가 필요하다

- ① 노드 안 (Pod ↔ 노드) — CNI 스펙이 정의하는 범위
 - kubelet → containerd → /opt/cni/bin/<플러그인> 실행 → Pod에 eth0 생성
- ② 노드 간 — 스펙이 정의하지 않는 범위
 - 구현체(Flannel / Calico / Cilium)의 DaemonSet이 담당
 - 방식: 터널 / 라우트 광고 / eBPF
- 쿠버네티스 네트워크 모델 요구: 모든 Pod가 NAT 없이 상호 도달 → ②가 반드시 필요
- "CNI"라는 용어의 두 의미
 - 스펙 (①의 호출 규약)
 - 구현체 (①+②를 모두 수행하는 소프트웨어)

1. CNI 스펙

1.1 정의 범위

스펙은 `containernetworking/cni` 저장소의 `SPEC.md` 단일 파일

정의하는 것	정의하지 않는 것
실행 파일 호출 규약 (환경변수 + stdin/stdout JSON)	네트워크 구현 방식
설정 파일 포맷 (<code>.conflist</code>)	노드 간 연결
결과 타입 (<code>interfaces</code> , <code>ips</code> , <code>routes</code> , <code>dns</code>)	IP 대역 관리 정책
오퍼레이션 (<code>ADD</code> / <code>DEL</code> / <code>CHECK</code> / <code>GC</code> / <code>STATUS</code> / <code>VERSION</code>)	NetworkPolicy

1.2 NetworkPolicy는 스펙 밖

- 쿠버네티스는 `NetworkPolicy` 리소스를 정의만 함
- **집행**은 CNI 구현체의 자발적 기능
- 스펙만 준수하는 플러그인의 동작
 - NetworkPolicy 리소스 무시
 - 생성해도 아무 일도 일어나지 않음. **에러 없음**
- Flannel 단독 구성이 NetworkPolicy를 지원하지 않는 이유

1.3 주요 오퍼레이션

오퍼레이션	도입	요건
<code>ADD</code>	0.1.0	네트워크 구성. 결과 JSON 반환
<code>DEL</code>	0.1.0	해제. 역등 . 대상 부재 시에도 성공 처리
<code>GC</code>	1.1.0	런타임이 유효 attachment 목록 전달 → 플러그인이 잔여 자원 정리
<code>STATUS</code>	1.1.0	<code>ADD</code> 처리 가능 상태인지 응답

- `GC` 도입 배경
 - `DEL` 미호출 시(kubelet 비정상 종료 등) 자원 누수
 - 2주차 `/var/lib/cni/networks/` 스테일 파일이 해당 사례
 - 종래에는 정리 규약 부재
- 에러 코드
 - `11` 일시적 실패 → 런타임 재시도
 - `7` 설정 오류 → 재시도 없이 Pod 생성 실패

2. 플러그인 분류

2.1 세 가지 타입

타입	역할	예
main	인터페이스 생성	bridge, ptp, macvlan, calico, cilium-cni
ipam	주소 할당	host-local, dhcp, static, calico-ipam
meta	기존 결과 변형·보강	portmap, bandwidth, tuning, firewall

- 역할 분리 → 조합 가능
 - bridge + host-local, bridge + dhcp
 - Calico: 인터페이스는 자체, portmap은 표준 플러그인 재사용

2.2 체인 내 위치

```
"plugins": [  
  { "type": "calico",    "ipam": {"type": "calico-ipam"}  
},    // main - 첫 번째  
  { "type": "portmap",  "capabilities": {"portMappings":  
true} }, // meta  
  { "type": "bandwidth", "capabilities": {"bandwidth": tr  
ue} }    // meta  
]
```

- main: 반드시 첫 번째, 단일
 - 두 번째 이후 플러그인은 prevResult를 받아 기존 인터페이스를 처리
 - 다중 인터페이스 필요 시 delegating plugin 사용
- ipam: 체인의 원소가 아님
 - main 설정 내부에 중첩. main이 실행 중 직접 호출
- meta: 개수 제한 없음
 - 완성된 결과에 iptables / tc 설정 추가

2.3 capabilities / runtimeConfig

	정적 설정	<code>capabilities</code> + <code>runtimeConfig</code>
값의 출처	confliست에 기재	런타임이 실행 시점에 주입
예	<code>"bridge": "cni0"</code>	Pod spec <code>hostPort</code> → <code>portMappings</code>
선언	일반 키	<code>"capabilities": {"portMappings": true}</code>

- `capabilities` = "해당 값을 런타임으로부터 받을 수 있다"는 선언
- 실제 값은 런타임이 stdin JSON의 `runtimeConfig` 키로 주입
- 미선언 플러그인에는 주입되지 않음 (조용히 무시)

```
Pod spec hostPort: 8080
→ containerd: portmap의 portMappings capability 확인
→ stdin JSON에 "runtimeConfig": {"portMappings": [...]} 추가
→ portmap: iptables DNAT 를 생성
```

2.4 delegating plugin

- 스펙 외 분류. 다른 플러그인을 대신 호출하는 플러그인
- **Multus** — primary CNI 자리에서 여러 플러그인에 위임. 다중 인터페이스 생성 (6주차)
- **flannel** 플러그인 — `subnet.env` 를 읽어 `bridge` 설정 생성 후 `bridge` 에 위임

3. Primary CNI

3.1 정의

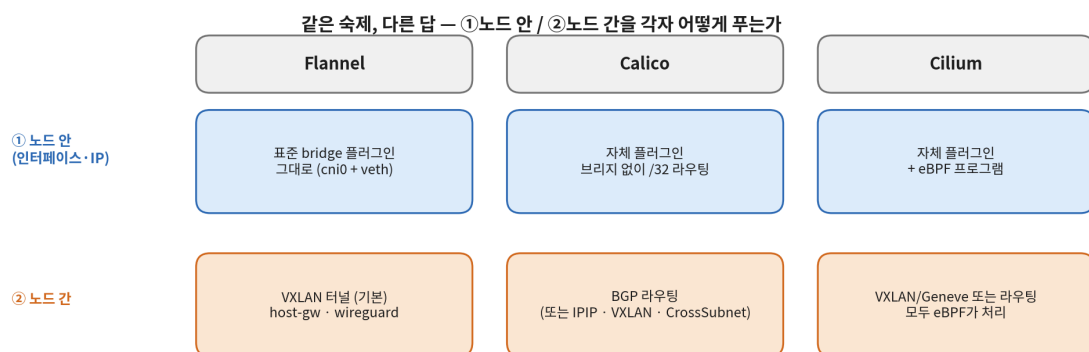
- 스펙 용어 아님. 실무 용어
- 조건
 1. Pod 기본 인터페이스 `eth0` 생성
 2. 클러스터 전역 Pod-to-Pod 도달성 확보 — 쿠버네티스 네트워크 모델 요구사항
 3. 통상 NetworkPolicy 집행
- `macvlan`, `ipvlan`, `host-device`, `sriov`: 인터페이스는 생성하나 2번 미담당
→ **secondary network**

3.2 노드 간 연결 세 가지 접근

접근	방식	대표
오버레이 (캡슐화)	Pod 패킷을 노드 간 터널에 적재. 물리망은 Pod IP 불필요	Flannel VXLAN, Calico IPIP/VXLAN
언더레이 (라우팅)	물리 라우터에 Pod CIDR 경로 광고	Flannel host-gw, Calico BGP
클라우드 네이티브 IPAM	Pod에 VPC 실제 주소 할당	AWS VPC CNI, Azure CNI

- 대가
 - 오버레이: 물리망 요구사항 없음 ↔ 캡슐화 오버헤드, **MTU 축소** (1500 → 1450)
 - 언더레이: 오버헤드 0, MTU 온전 ↔ **물리망 협조 필요** (동일 L2 또는 BGP 피어링)
 - 클라우드 네이티브: 성능·통합 최상 ↔ **VPC IP 소모**, 노드당 ENI/IP 상한

4. 구현체 구조



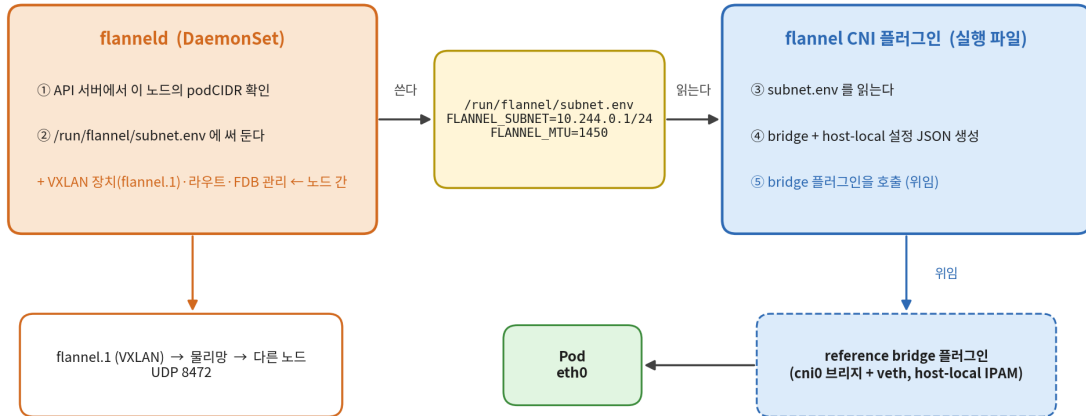
세 구현체 모두 § 3.1의 Primary CNI 조건(eth0 생성 + 클러스터 전역 도달성)을 채운다. 다른 건 방법뿐이다.

- 세 구현체의 차이는 ① / ②의 해결 방식

4.1 Flannel

설계 철학: 최소한만 구현. 노드 간 연결 외에는 표준 플러그인에 위임

Flannel — flanneld와 CNI 플러그인은 파일 하나로 느슨하게 연결된다



핵심: 노드 안 데이터패스 = 표준 bridge 플러그인 그대로. Flannel 고유의 것은 노드 간(백엔드)뿐이다.

컴포넌트	역할
<code>flanneld</code> (DaemonSet)	노드별 서브넷 확보, 백엔드 설정, 라우트/FDB 관리
<code>/run/flannel/subnet.env</code>	flanneld가 쓰고 CNI 플러그인이 읽는 파일 인터페이스
<code>flannel</code> CNI 플러그인	<code>subnet.env</code> 기반으로 <code>bridge</code> + <code>host-local</code> 설정 생성 후 위임
데이터스토어	쿠버네티스 API (<code>node.spec.podCIDR</code>) 또는 etcd

```
$ cat /run/flannel/subnet.env
FLANNEL_NETWORK=10.244.0.0/16
FLANNEL_SUBNET=10.244.0.1/24
FLANNEL_MTU=1450
FLANNEL_IPMASQ=true
```

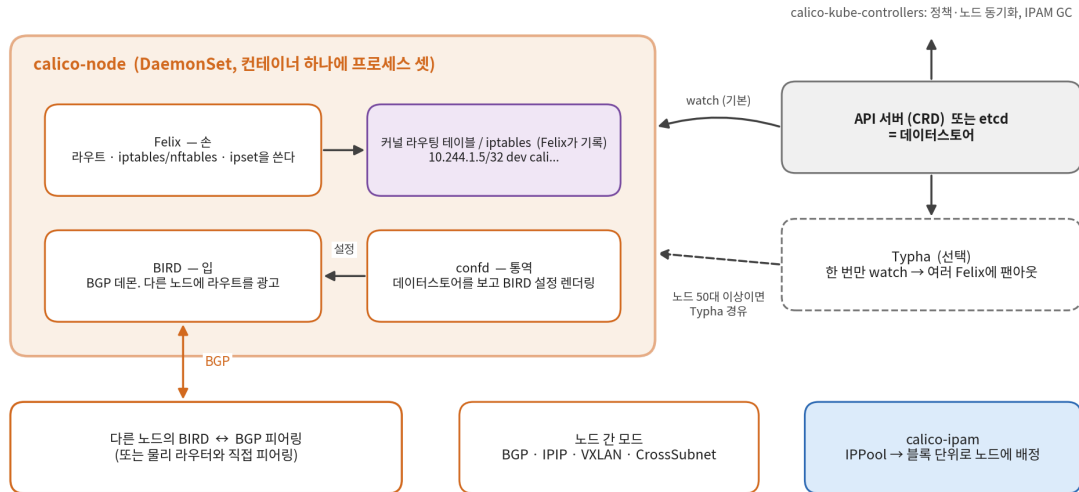
```
{ "type": "bridge", "bridge": "cni0", "isGateway": true, "mtu": 1450, "ipam": { "type": "host-local", "subnet": "10.244.0.1/24" } }
```

- 노드 안 데이터패스 = reference `bridge` 플러그인. Flannel 고유 영역은 백엔드(노드 간)뿐

4.2 Calico

설계 철학: 표준 라우팅 프로토콜로 해결. 각 노드가 라우터

Calico — 각 노드가 라우터, 각 Pod가 /32 경로, 노드끼리는 BGP로 광고



브리지 없음. Pod의 veth가 노드에 직접 붙고, 노드가 /32 라우팅으로 전달한다.

컴포넌트	역할
Felix (calico-node 내)	데이터플레인. 라우트·iptables/nftables-ipset 프로그래밍
BIRD (calico-node 내)	BGP 데몬. 노드 간 라우트 광고
confd (calico-node 내)	데이터스토어 감시 → BIRD 설정 렌더링
calico-ipam	자체 IPAM (IPPool → IPAMBlock → BlockAffinity)
Typha (선택)	대규모 클러스터에서 API 서버 watch 부하 흡수. 통상 노드 50대 이상
calico-kube-controllers	정책·노드 오브젝트 동기화, IPAM GC
데이터스토어	쿠버네티스 CRD (기본) 또는 etcd

모드 — 두 축

축	선택지
노드 간 (②)	BGP (no overlay) / IPIP (tunl0) / VXLAN (UDP 4789) / CrossSubnet
노드 안 (①)	iptables (기본) / nftables / eBPF (kube-proxy 대체 가능)

- CrossSubnet: 목적지 노드가 동일 서브넷이면 직결, 아니면 캡슐화

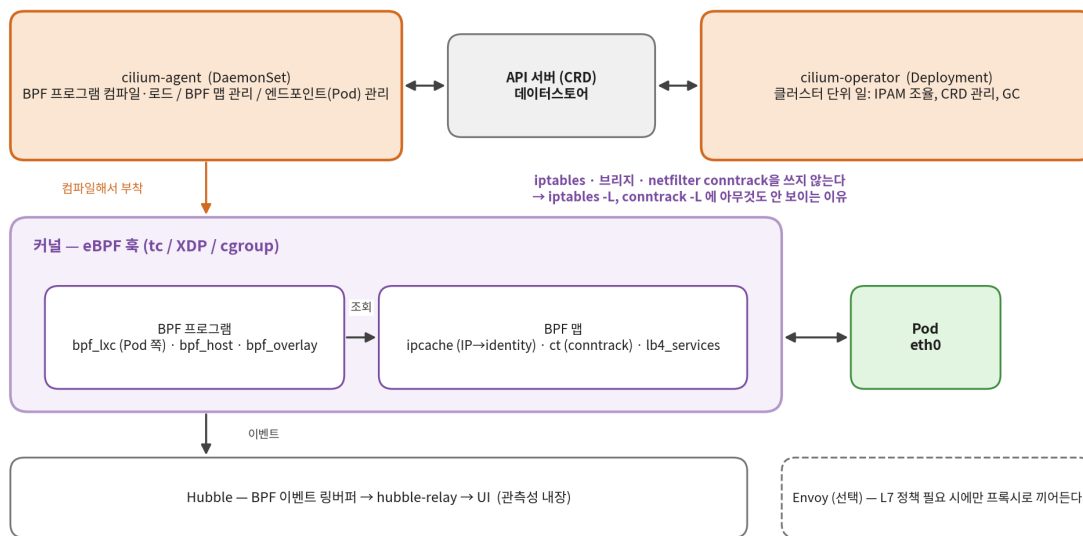
특징

- NetworkPolicy 집행 + 자체 확장 (`GlobalNetworkPolicy` , 정책 순서, DNS 정책)
- 브리지 없는 `/32` 라우팅 모델
- BGP로 물리 패브릭과 직접 피어링 → Pod IP를 DC 전역에 라우팅 가능

4.3 Cilium

설계 철학: 커널 네트워크 스택을 eBPF로 대체

Cilium — 커널 네트워크 스택을 우회하고 eBPF 프로그램이 패킷을 처리한다



컴포넌트	역할
cilium-agent (DaemonSet)	BPF 프로그램 컴파일·로드, 맵 관리, 엔드포인트 관리
cilium-operator	클러스터 단위 작업 — IPAM 조율, CRD 관리, GC
BPF 프로그램	<code>bpf_lxc</code> , <code>bpf_host</code> , <code>bpf_overlay</code> — tc/tcx/XDP/cgroup 쪽에 부착
BPF 맵	<code>cilium_ipcache</code> , <code>cilium_ct4_global</code> , <code>cilium_lb4_services_v2</code> 등
Hubble	관측성. agent 이벤트 링버퍼 → hubble-relay → UI
Envoy (선택)	L7 정책 집행 시 프록시

Identity

- Pod 레이블 조합 → 숫자 ID 파생
- 정책은 IP가 아닌 identity 간 규칙으로 기술
- Pod 재생성으로 IP가 바뀌어도 정책 불변

차별점

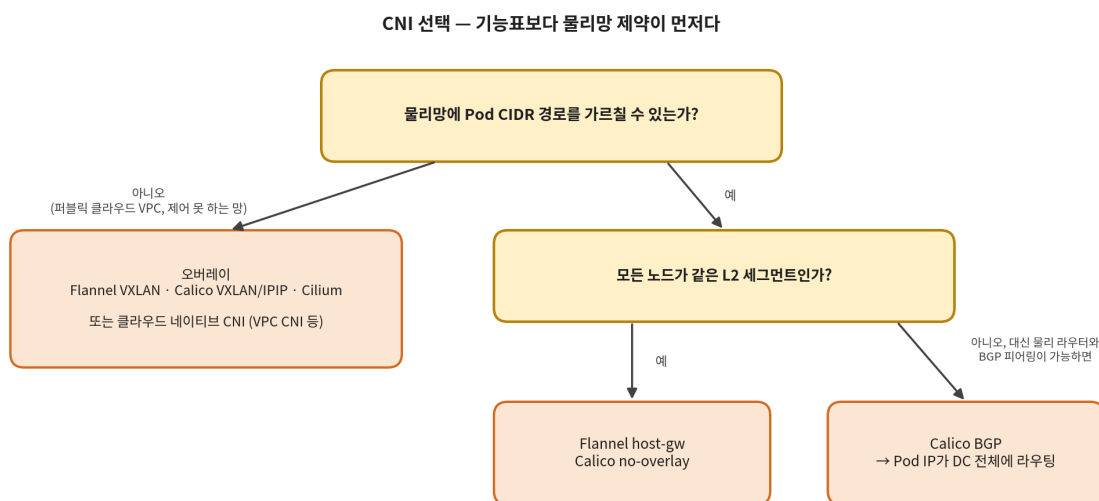
항목	Flannel / Calico (기본)	Cilium
Service 구현	kube-proxy (iptables/IPVS)	BPF 맵 조회 — kube-proxy 제거 가능
정책 대상	IP/CIDR	Identity
정책 계층	L3/L4	L3/L4 + L7 (HTTP/gRPC/Kafka/DNS)
관측성	별도 도구	Hubble 내장
conntrack	커널 netfilter	자체 BPF 맵

4.4 기타 구현체

구현체	특징
AWS VPC CNI / Azure CNI	클라우드 네이티브. Pod에 VPC 실주소
GKE Dataplane V2	Cilium 기반
Kube-OVN / Antrea	OVS 기반. VPC·서브넷·QoS 등 가상 네트워크 기능
kindnet	kind용 최소 구현. 학습용
Weave Net	유지보수 종료

5. 선택 기준

5.1 물리망 제약 우선



환경에서 가능한 것을 먼저 좁힌 뒤, 그 안에서 요구사항(NetworkPolicy · L7 · BGP 통합)으로 고른다. 교체는 비싸다.

- 기능 비교보다 **환경 제약**이 선행
- 첫 질문: 물리망에 Pod CIDR 경로를 가르칠 수 있는가

5.2 요구사항별

요구사항	선택
단순, 학습·소규모	Flannel VXLAN
NetworkPolicy	Calico, Cilium (Flannel 단독 불가)
물리망 BGP 통합	Calico
L7 정책 / 관측성 / kube-proxy 제거	Cilium
클라우드 VPC 통합	해당 클라우드 CNI
다중 인터페이스 (CNF/telco)	Multus + secondary (6주차)

5.3 운영 비용

	Flannel	Calico	Cilium
학습 곡선	낮음	중간	높음
트러블슈팅	커널 도구로 전부 관측 가능	<code>calicoctl</code> + iptables/ipset	커널 도구로 불가. <code>cilium</code> CLI 필수

- Cilium: 패킷이 커널 netstack 우회 → `iptables -L`, `conntrack -L` 에 미표시
- `cilium monitor`, `cilium bpf ct list` 등 전용 도구 필요

5.4 교체 비용

- **CNI 교체는 무중단 곤란**. Pod IP 대역과 데이터패스 전면 변경
- 통상 방법
 - 신규 클러스터 구성 후 워크로드 마이그레이션 (권장)
 - 노드 순차 drain → 교체 → 재조인 (다운타임 구간 존재)
- → **초기 선택의 비중이 큼**. 제약과 요구사항을 먼저 확정